Aonde você quer chegar?

Vai com a

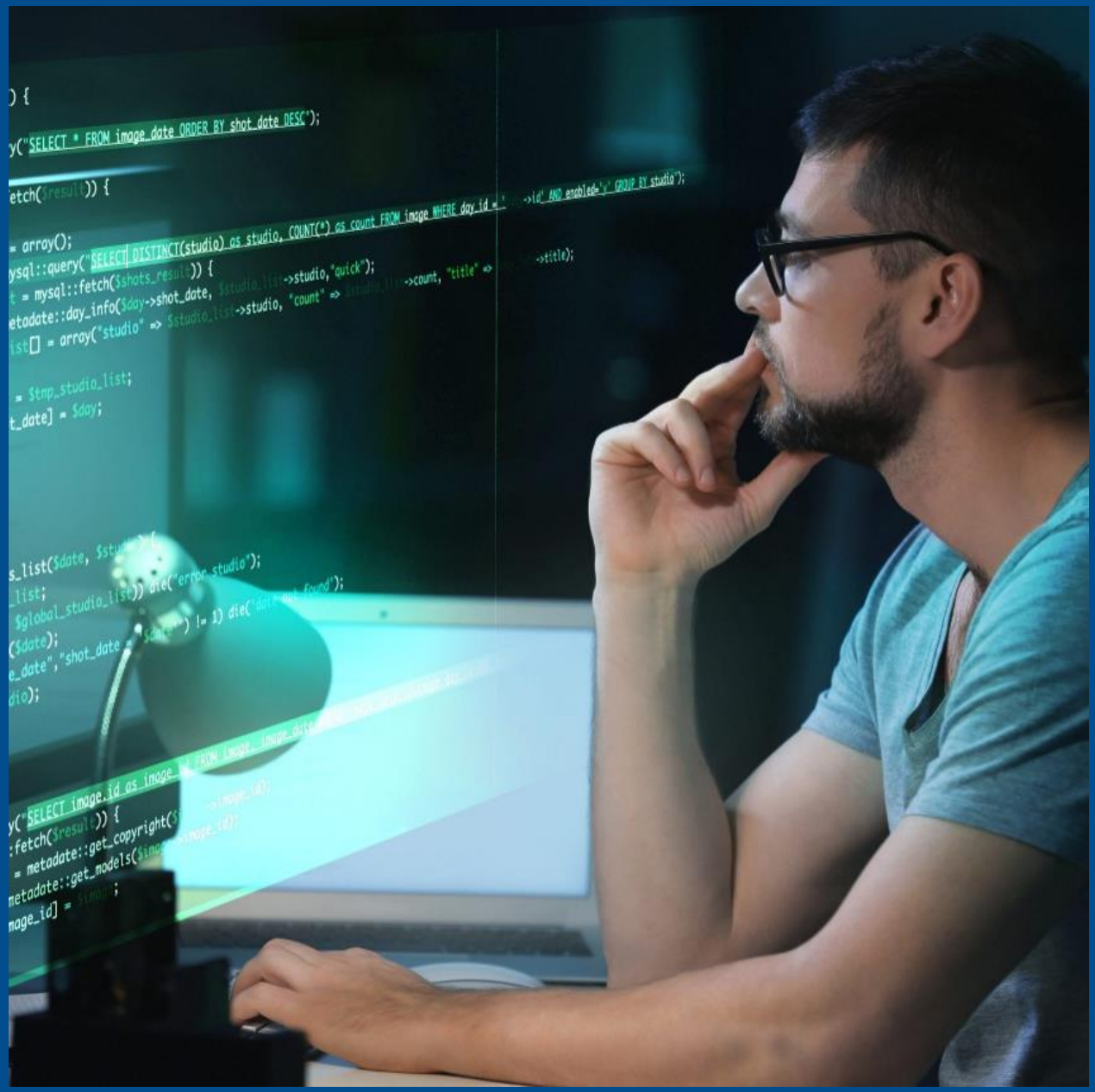
UniCesumar

EAD, SEMIPRESENCIAL E PRESENCIAL

UniCesumar
EAD, SEMIPRESENCIAL E PRESENCIAL

Tecnologias Emergentes em Engenharia de *Software*

Aula 2



Prof. Rômulo Santos

PhD Computação. MSc Engenharia Elétrica. MSc Ciência da Informação.

Sumário

- Modelos Organizacionais em Fábricas de *Software*.
- Fábrica de *Software* Orientada a Processos, Produtos, Projetos e Programas.
- Como escolher e combinar modelos?
- Impactos na gestão de equipes e papéis.
- Ciclos de desenvolvimento e ferramentas.

Modelos Organizacionais em Fábricas de *Software*:

O **modelo organizacional** define como uma fábrica de *software* estrutura suas decisões, prioridades, fluxos de trabalho e indicadores de desempenho. Muitas dificuldades operacionais não decorrem apenas de ferramentas ou métodos, mas de um **arranjo organizacional inadequado ao contexto**.

Quatro modelos são recorrentes:

- **Orientado a Processos:** prioriza **padronização, controle, qualidade e previsibilidade**;

- **Orientado a Produtos:** enfatiza **evolução contínua, reuso, valor ao usuário e ciclo de vida do *software***;

- **Orientado a Projetos:** organiza o trabalho em torno de **entregas, prazos, escopo e marcos definidos**;

- **Orientado a Programas:** coordena **múltiplos projetos e iniciativas relacionadas**, buscando alinhamento estratégico.

Na prática, esses modelos podem **coexistir em uma mesma organização**. Empresas reguladas, como órgãos públicos e instituições financeiras, tendem a valorizar maior controle por processos e programas, enquanto ***software houses* e *fintechs*** frequentemente combinam estruturas orientadas a produtos e projetos.

Fábrica de *Software* Orientada a Processos:

A **fábrica orientada a processos** organiza o trabalho por meio de **fluxos padronizados**, com entradas, atividades, critérios de validação e resultados claramente definidos. Seu principal objetivo é aumentar a **previsibilidade, rastreabilidade e qualidade** das entregas.

Esse modelo é especialmente adequado quando há:

- **Alto volume de demandas repetitivas**;
- Necessidade de atendimento a **SLAs**;
- Exigência de **rastreabilidade e auditoria**;
- Processos que precisam apresentar resultados consistentes.

Exemplos:

Nova API: contrato inicial → testes de contrato aprovados + documentação publicada;

Evolução de tela: protótipo e critérios de aceite → testes executados + release notes;

Correção P1: evidência do defeito → *hotfix* validado + *rollback* testado.

Fábrica de *Software* Orientada a Processos:

A graça do catálogo é poder medir o que importa sem esforço extra: observar o *lead time* por serviço (P50/P90), o percentual cumprido no SLA e a taxa de reabertura já conta a história do fluxo.

Há um cuidado que não pode faltar: evitar o "teatro de processo". Se algum *gate* não reduz risco nem encurta caminho, ele atrapalha, por isso, uma revisão trimestral para eliminar o que não agrega mantém o processo vivo e honesto (Albino; Aguiar, 2017).

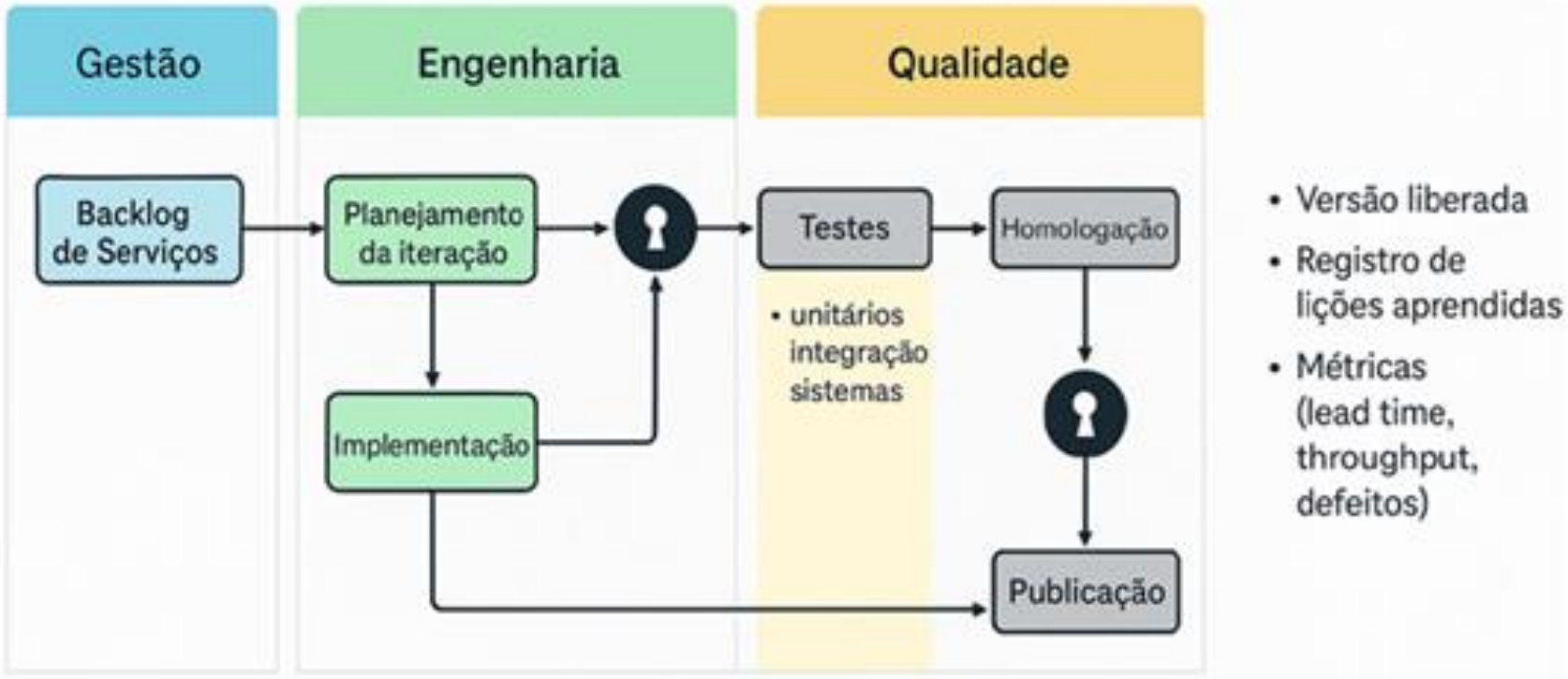


Figura 1 – Fluxo padrão orientado a processos: do backlog de serviços aos *gates* (revisão + testes + publicação)
Fonte: gerado por ChatGPT em 26 set. 2025.

Fábrica de *Software* Orientada a Produtos:

No arranjo por produtos, o centro é a evolução contínua de plataformas e serviços. Para evitar “copiar e colar”, bem como o caos de dependências, ado taremos arquitetura de referência, componentes reutilizáveis versionados e variabilidades mapeadas (linha de produtos).

A operação fica mais fluida quando contratos de API/eventos são executáveis (testes de contrato) e *changelogs* orientam migrações (ABNT, 2009; Softex, 2016a). Empresas SaaS brasileiras obtêm ganhos diretos de tempo e qualidade ao tratar reuso como produto interno.

KIT DE PRODUTO
Visão, *guidelines* de arquitetura, catálogo de componentes e templates de serviço.
GOVERNANÇA LEVE
Janelas de depreciação e migração guiada por *playbooks*.
KPIS DE PRODUTO
Adoção de componentes, efeitos por versão, *cycle time* por épica, *lead time* de evolução.

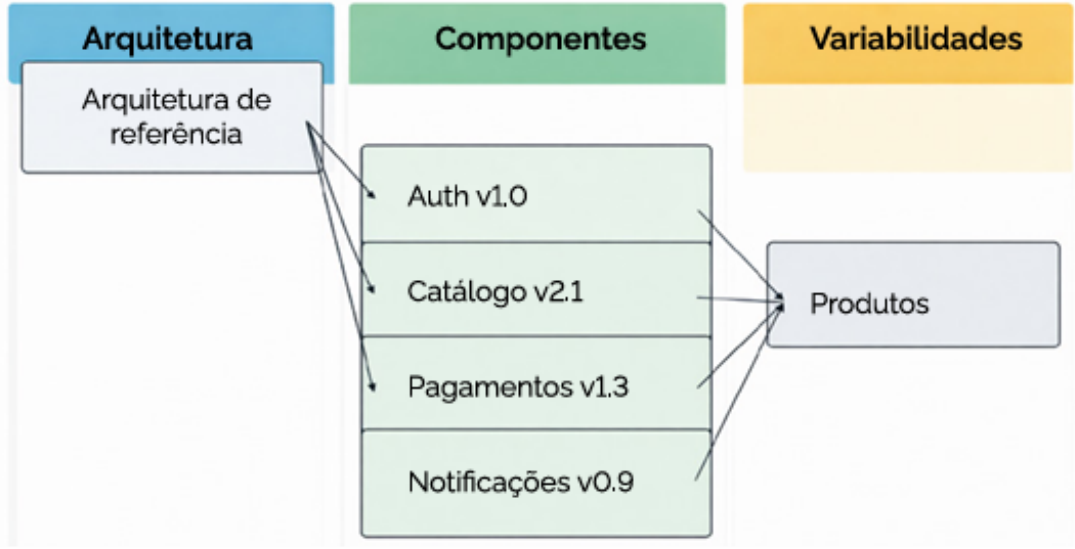


Figura 2 – Mapa de produto e linha de produtos (LPS): arquitetura de referência, componentes reutilizáveis, variabilidades e políticas comuns / Fonte: gerado por ChatGPT em 26 set. 2025.

Fábrica de *Software* Orientada a Projetos:

Sobre as entregas pontuais, marcos e integração com a esteira comum, projetos têm escopo/cronograma definidos (migração de *gateway*, adequação fiscal, *rollout* regional).

O risco é cair no *big bang*. O caminho brasileiro mais sólido tem sido combinar marcos com entregas incrementais conectadas à esteira padronizada: homologação, *feature flags*, *cutover* com *roll-back* (TCU, 2018; ABNT, 2009).

EAP LEVE + MARCOS

Com critérios de aceite e verificação objetiva.

INTEGRAÇÃO À ESTEIRA CI/CD COMUM

Garantindo padronização dos processos de *build*, testes e *deploy dress rehearsal* de *cutover*.

KPIS DE PROJETO

Cumprimento de marcos, desvio de prazo/custo, defeitos por etapa.

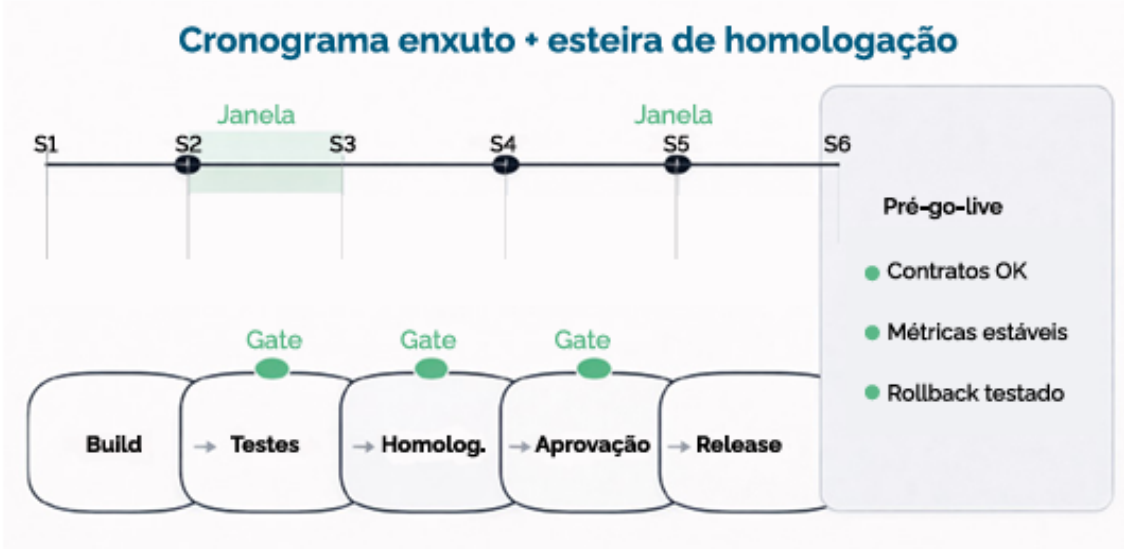


Figura 3 – Cronograma enxuto + esteira de homologação: *milestones*, janelas de liberação e checagens pré-*go-live* / Fonte: gerado por ChatGPT em 26 set. 2025.

Fábrica de *Software* Orientada a Programas:

Vamos tratar das dependências, priorização por valor/risco e métricas de portfólio?

Programas coordenam frentes interdependentes (experiência do cidadão, *open finance*, interoperabilidade entre órgãos).

A governança eficaz decide alocação por valor e risco, remove bloqueios e torna visíveis as dependências (TCU, 2014; Brasil, 2018). O objetivo não é criar camadas de comitês, mas sincronizar cadências.

RITUAIS DE SINCRONIZAÇÃO (QUINZENAL/MENSAL)

Focados em **bloqueios** e **apostas** (não em status).

MAPA DE DEPENDÊNCIAS E ORÇAMENTO DE RISCO

Por exemplo: improváveis/impacto alto → planos de contingência.

KPIS DE PROGRAMA

Throughput do portfólio, dependências resolvidas por ciclo, alinhamento a objetivos estratégicos.

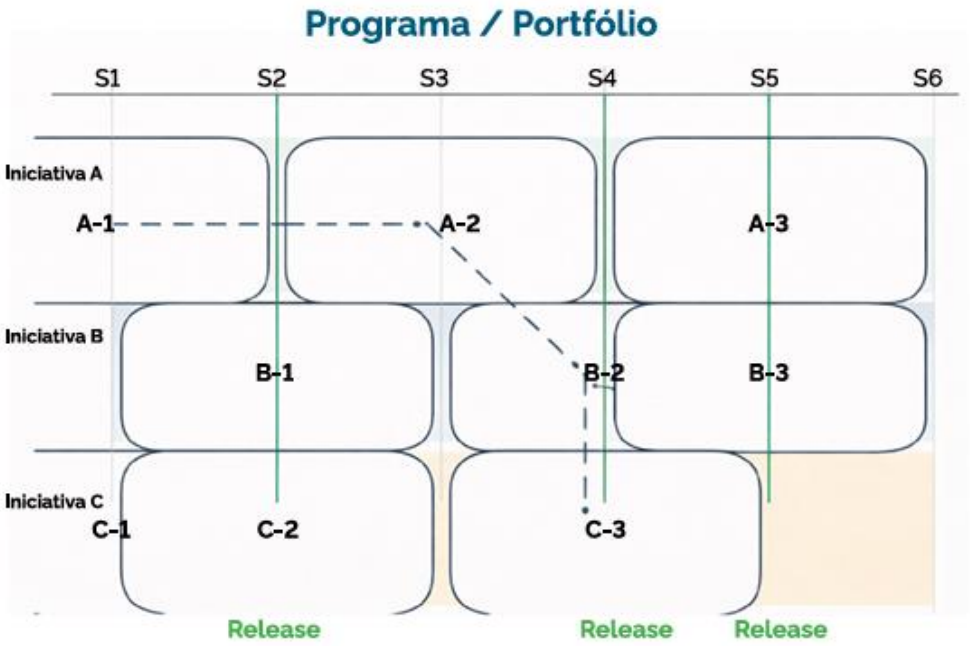


Figura 4 – Quadro de programa/portfólio: iniciativas, dependências e cadências
Fonte: gerado por ChatGPT em 26 set. 2025.

ESTUDO DE CASO

FÁBRICA DE SOFTWARE EM UMA ORGANIZAÇÃO BRASILEIRA

Como combinar processos, produtos, projetos e programas utilizando uma mesma esteira de desenvolvimento

Objetivo:
Entregar soluções digitais com qualidade, previsibilidade e alinhamento estratégico

Cenário

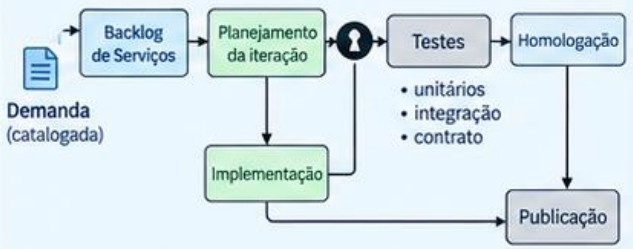
Uma organização pública com múltiplos serviços digitais precisa modernizar seus sistemas, criar uma plataforma comum e atender exigências regulatórias, ao mesmo tempo em que coordena uma transformação digital. A fábrica de *software* adota diferentes modelos organizacionais (Processos, *Produtos*, *Projetos* e *Programas*), todos integrados a uma mesma esteira CI/CD e governança leve.

1. PROCESSOS

Serviços digitais para atendimento contínuo

A organização padroniza a entrada e execução das demandas de serviços, utilizando catálogo de serviços, esteiras padronizadas e *quality gates*.

Exemplo: Secretaria Estadual de Saúde



Resultados:

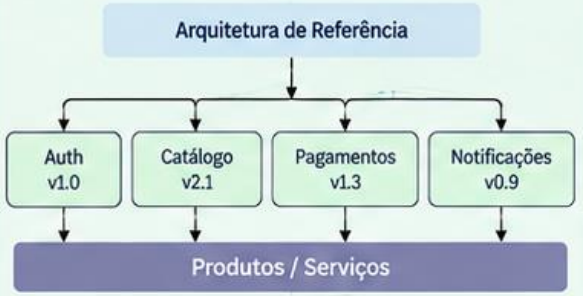
- SLA passou de 62% para 89%
- Rastreabilidade automática para auditorias
- Redução de retrabalho

2. PRODUTOS

Plataformas com evolução contínua

Criação de uma plataforma de serviços com componentes reutilizáveis, arquitetura de referência e roadmap de evolução.

Exemplo: Banco / Fintech



Resultados:

- Reuso de componentes
- Redução de defeitos por release
- Menor *lead time* nas entregas

3. PROJETOS

Iniciativas com prazo, escopo e marcos

Tratamento de demandas específicas (ex.: obrigação regulatória) com planejamento detalhado, marcos e entregas incrementais.

Exemplo: Adequação regulatória



Resultados:

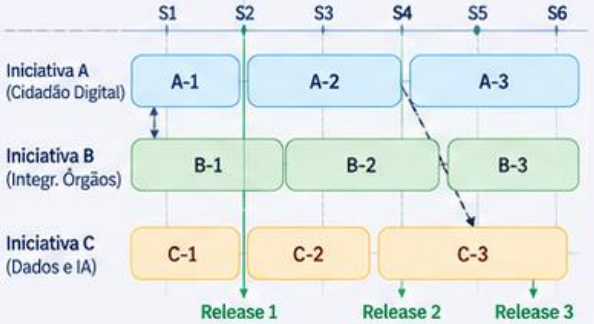
- Cumprimento de marcos regulatórios
- Menor risco de *big bang*
- Integração com processos e produtos

4. PROGRAMAS

Múltiplas iniciativas interdependentes

Coordenação de várias frentes que precisam convergir para um objetivo estratégico comum.

Exemplo: Transformação Digital



Resultados:

- Visão integrada do portfólio
- Gestão de dependências
- Entrega de valor estratégico

ESTEIRA CI/CD COMUM (AUTOMAÇÃO E QUALIDADE)

Código (Commit) → Lint e Análise Estática → Build → Testes Unitários → Testes de Contrato → Publicação de Artefatos → Deploy (com Feature Flags) → Verificação Pós-Deploy

RISCOS (quando desequilibrado)

- Processos em excesso → silos e burocracia
- Produto sem governança → volatilidade
- Projeto sem integração → soluções fragmentadas
- Programa sem foco → aumento da complexidade

BOAS PRÁTICAS

- Combinar modelos conforme o tipo de trabalho
- Governança leve e baseada em evidências
- Automação e qualidade integradas à esteira
- Métricas simples e relevantes (SLA, *lead time*, *throughput*, defeitos)
- Ajuste contínuo da estrutura organizacional

RESULTADO FINAL

- Entregas mais rápidas e previsíveis
- Melhor qualidade e menor retrabalho
- Conformidade regulatória e auditabilidade
- Reuso e evolução contínua das soluções
- Alinhamento com os objetivos estratégicos da organização

Como escolher e combinar modelos?

A escolha do modelo organizacional deve partir do **tipo de trabalho realizado**, e não de tendências ou preferências por determinado método. Processos, Produtos, Projetos e Programas são **formas diferentes de organizar o trabalho**, cada uma adequada a um contexto específico.

Processos: indicados para atividades repetitivas, padronizáveis e auditáveis;

Produtos: adequados a soluções que exigem evolução contínua, *roadmap* e reutilização;

Projetos: apropriados para iniciativas com prazo, custo, escopo e marcos claramente definidos;

Programas: utilizados para coordenar múltiplas iniciativas interdependentes em torno de um objetivo estratégico comum.

No contexto de Engenharia de *Software*, uma mesma organização pode utilizar os quatro modelos simultaneamente.

Como escolher e combinar modelos?

Uma forma simples de selecionar o modelo é observar as características predominantes da demanda.

Alto volume repetitivo e auditável → Processos

Ideal para filas contínuas de atendimento, correções conhecidas, solicitações padronizadas e esteiras operacionais. Indicadores importantes: ***lead time*, SLA, *throughput* e variabilidade**.

Plataforma com evolução contínua → Produtos

Adequado quando existe uma solução que evolui permanentemente, como aplicativos, plataformas de dados ou sistemas centrais. O foco está em ***roadmap*, valor entregue, reutilização e evolução contínua**.

Data fixa ou exigência regulatória → Projetos

Aplicável quando existem marcos inegociáveis, orçamento definido e critérios claros de aceitação. Exige atenção a **cronograma, caminho crítico, riscos e mudanças de escopo**.

Múltiplas frentes interdependentes → Programas

Indicado quando diferentes projetos ou *squads* precisam convergir para um mesmo resultado estratégico, exigindo ***roadmap* integrado, governança e gestão de dependências**.

Como escolher e combinar modelos?

O modelo organizacional não precisa ser permanente. Alguns sinais indicam que a estrutura atual não está respondendo adequadamente às necessidades do trabalho.

***Lead time* instável e filas desorganizadas:** fortalecer uma abordagem por **Processos**, com padrões, limites de WIP e SLAs;

Retrabalho e duplicação entre equipes: adotar uma visão de **Produto**, criando plataformas e componentes reutilizáveis;

Marcos obrigatórios ou prazos críticos em risco: tratar a iniciativa como **Projeto**, utilizando baseline, cronograma e gestão formal de mudanças;

Dependências entre *squads* causando bloqueios: elevar a coordenação para **Programa**, com *roadmap* integrado e gestão de interdependências.

A combinação de modelos também pode ser necessária. Uma organização pode operar serviços por **Processos**, desenvolver uma plataforma como **Produto**, realizar uma migração regulatória como **Projeto** e coordenar uma transformação digital como **Programa**.

Estudo de Caso: Como Escolher e Combinar Modelos?

Aplicação prática em uma fábrica de software educacional

Cenário

Uma instituição de ensino precisa **operar serviços digitais recorrentes, evoluir continuamente sua plataforma acadêmica, cumprir um prazo regulatório para novo portal de matrícula** e coordenar várias frentes de transformação digital ao mesmo tempo.

A liderança precisa decidir **qual modelo organizacional** usar em cada tipo de demanda.



Processos

Operação eficiente e confiável

Quando usar
- Alto volume repetitivo e auditável.

No caso
- Chamados de suporte, correções padrão e solicitações de rotina.

Práticas
- Catálogo de serviços, SLA, limites de WIP, fluxo padronizado.

Resultado
- *Lead time* mais estável e maior previsibilidade.

Produtos

Evolução contínua e geração de valor

Quando usar
- Plataforma com evolução contínua.

No caso
- App do aluno, portal acadêmico e componentes reutilizáveis.

Práticas
- Roadmap, *product ownership*, arquitetura de referência, versionamento.

Resultado
- Menos retrabalho e mais reuso.

Projetos

Entrega com prazo e escopo definidos

Quando usar
- Data fixa ou exigência regulatória.

No caso
- Novo portal de matrícula com prazo definido.

Práticas
- Marcos, cronograma, baseline, gestão de riscos e mudanças.

Resultado
- Entrega no prazo com escopo controlado.

Programas

Coordenação estratégica de múltiplas iniciativas

Quando usar
- Múltiplas frentes interdependentes.

No caso
- Transformação digital integrando secretaria, financeiro, biblioteca e TI.

Práticas
- *Roadmap* integrado, governança leve, gestão de dependências.

Resultado
- Alinnhamento estratégico e coordenação entre *squads*.

Como combinar no mesmo contexto

Serviços recorrentes (Processos) → **Plataforma acadêmica** (Produto) → **Portal com prazo regulatório** (Projeto) → **Transformação institucional** (Programa)

Aprendizado

Não existe modelo único. Organizações maduras combinam **Processos, Produtos, Projetos** e **Programas** conforme o tipo de trabalho, os riscos e o objetivo estratégico.
O segredo está em ajustar a estrutura quando os sinais do contexto mudam.

Impactos na gestão de equipes e papéis:

O **modelo organizacional** influencia diretamente a distribuição de responsabilidades, a autonomia das equipes, os *handoffs* e a carga cognitiva dos profissionais.

Processos: exigem responsáveis por catálogos de serviços, *quality gates* e métricas de fluxo;

Produtos: fortalecem o papel de ***Product Ownership***, arquitetura de referência e práticas de versionamento;

Projetos: demandam controle claro de **riscos, prazos, escopo e marcos**;

Programas: coordenam dependências entre equipes e direcionam esforços para objetivos estratégicos comuns.

Em todos os modelos, o desafio é manter uma **governança leve**, priorizando evidências objetivas, *dashboards* simples e informações produzidas automaticamente pelas próprias ferramentas e esteiras de desenvolvimento.

Ciclos de desenvolvimento e ferramentas:

Os **ciclos de desenvolvimento** devem ser escolhidos de acordo com o tipo de trabalho e com o modelo organizacional adotado. Não é necessário utilizar uma única abordagem para toda a fábrica de *software*.

Processos: favorecem ciclos curtos, repetitivos e quase contínuos;

Produtos: evoluem a partir de ***roadmaps*, épicas e entregas incrementais**;

Projetos: combinam marcos definidos com incrementos para reduzir o risco de uma entrega única ao final;

Programas: utilizam momentos de sincronização para coordenar múltiplas equipes e dependências.

Independentemente da abordagem, **Cascata, Ágil ou Híbrida,** a automação da engenharia de software é fundamental. Uma esteira **CI/CD** pode integrar etapas como:

Lint → Build → Testes Unitários → Testes de Contrato → Publicação → Deploy → Verificação Pós-Deploy

Estudo de Caso 1: Setor Público

Padronização de serviços digitais em uma Secretaria Estadual de Saúde

Cenário

Uma secretaria estadual de saúde precisava centralizar a entrada das demandas digitais, reduzir retrabalho e melhorar o atendimento aos órgãos internos. As solicitações chegavam de formas diferentes, sem padrão claro de priorização e acompanhamento.

Modelo aplicado

A organização adotou uma **fábrica de software orientada a Processos**, com os seguintes elementos:

Catálogo de serviços
Serviços bem definidos e critérios de entrada.

Esteira padronizada
Fluxo único de ponta a ponta para todas as demandas.

Quality gates leves
Pontos de controle de qualidade no fluxo.

Evidências automáticas
Registros e artefatos gerados automaticamente para auditoria.

Ações realizadas

1. Criação de catálogo de serviços e critérios de entrada.
2. Padronização do fluxo: *backlog* → planejamento → implementação → testes → homologação → publicação.
3. Adoção de SLAs e métricas de fluxo.
4. Registro automático de evidências para auditoria.

Fluxo da fábrica de software (pipeline)



Resultados

- SLA atendido subiu de 62% para 89%.
- Maior previsibilidade no atendimento.
- Rastreabilidade para auditorias.
- Redução de retrabalho e pedidos difusos.

Aprendizado

Quando o trabalho é repetitivo e auditável, o **modelo orientado a Processos** aumenta previsibilidade, qualidade e controle sem gerar burocracia excessiva.

Estudo de Caso 2: Banco / Fintech

Reuso de componentes e adequação regulatória com foco em Produtos e Projetos

Cenário

Um banco digital precisava evoluir sua plataforma de serviços, ampliar o reuso entre *squads* e atender rapidamente a novas exigências regulatórias. As equipes já desenvolviam soluções semelhantes, gerando duplicação de esforço, retrabalho e risco de inconsistências.

Modelo aplicado

A organização combinou uma **fábrica de software orientada a Produtos** com a **gestão por Projetos** para demandas regulatórias, utilizando:

- **Componentes versionados**
  Módulos estáveis com changelog e controle de versões.
- **Arquitetura de referência**
  Padrões comuns para integração e evolução da plataforma.
- **Roadmap do produto**
  Priorização contínua de funcionalidades e melhorias.
- **Projeto regulatório incremental**
  Marcos, cronograma e entregas parciais para conformidade.

Ações realizadas

1. Publicação de **componentes** reutilizáveis (auth, pagamentos, notificações).
2. Adoção de versionamento semântico e changelog claro.
3. Criação de roadmap da plataforma compartilhada.
4. Tratamento da obrigação regulatória como projeto com entregas incrementais.

Plataforma e entrega incremental



Resultados

- Maior reuso entre equipes.
- Redução do retrabalho e dos defeitos por release.
- Menor risco de big bang regulatório.
- Mais previsibilidade nas entregas.

Aprendizado

Quando a organização combina **Produto** para evolução contínua e **Projeto** para exigências com prazo definido, ela aumenta o reuso, reduz riscos e entrega conformidade com maior previsibilidade.

Estudo de Caso 3: Empresa SaaS

Combinação de Produtos e Processos para reduzir defeitos e acelerar entregas

Cenário

Uma empresa SaaS precisava manter a evolução contínua do seu produto, reduzir defeitos em produção e dar previsibilidade às demandas rotineiras de suporte e pequenas melhorias.
O crescimento da base de clientes exigia mais padronização sem engessar a inovação.

Modelo aplicado

A organização combinou uma **fábrica de software orientada a Produtos** com práticas de **Processos** para demandas recorrentes, adotando:

- **Componente de autenticação**
  Módulo reutilizável e versionado entre aplicações.
- **Roadmap contínuo**
  Backlog do produto guiado por valor e evolução.
- **Catálogo de serviços**
  Entrada padronizada para chamados e solicitações de rotina.
- **SLAs e métricas**
  Lead time, defeitos por release e previsibilidade operacional.

Ações realizadas

1. Reutilização do componente de autenticação em diferentes módulos.
2. Versionamento dos componentes com documentação e *changelog*.
3. Atendimento das demandas rotineiras por catálogo e SLA.
4. Uso de esteira CI/CD com testes e publicação contínua.

Fluxo de evolução e operação



Resultados

- Redução de defeitos por release.
- Menor *lead time* mediano.
- Maior previsibilidade nas rotinas de atendimento.
- Evolução contínua sem bloquear a inovação.

Aprendizado

Quando **Produto** e **Processos** coexistem de forma equilibrada, a empresa ganha escala operacional, reduz defeitos e mantém a evolução contínua do software.

Bons estudos!

Siga em frente com a



www.unicesumar.edu.br